MINI ASSESSMENT

API Handling & JSON Safety

NAME: P. Prem Kiran

UNIVERSITY: KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

REG. NO.: 99230040709

Assessment Questions & Answers

1. If an API returns 429, what should your program do?

Answer:
HTTP 429 means “Too Many Requests.” The program has reached the API's rate limit. It should wait before sending another request, and use Retry-After if the server provides it.

Simple comparison:

✗ **Bad approach**
Keep sending requests immediately → more 429 errors.

✓ **Better approach**
Wait → retry after the delay → stop after limited retries.

Example:

```
if response.status_code == 429:
    time.sleep(5)
    # retry after waiting
```

2. If response.json() fails, what should your program do?

Answer:
response.json() converts the response into JSON. It may fail when the server returns invalid JSON, HTML, plain text, or an empty response. Catch the error and handle it safely.

Simple comparison:

✗ **Unsafe approach**
Call response.json() without handling errors → program may crash.

✓ **Safer approach**
Use try/except → show an error or use a safe fallback.

Example:

```
try:
    data = response.json()
except ValueError:
    print("Invalid JSON response")
```

3. If a JSON key is missing, why might .get() be safer than []?

Answer:
Using [] requires the key to exist. A missing key causes KeyError. .get() is safer because it can return None or a default value, allowing the program to continue.

Simple comparison:

Using []	Using .get()
data["name"]	data.get("name")
Missing key → KeyError	Missing key → None / default value

Real-world example:

```
user = {"name": "Ganesh"}
city = user.get("city", "Unknown")
print(city)
```

Output: Unknown

Quick Revision

- 429 → Too Many Requests → wait and retry carefully.
- response.json() fails → catch the error and handle the response safely.
- Missing JSON key → .get() can return a default value instead of raising KeyError.